

NBA Pronos - Donnees stats

Dossier Cadrage/Stats/ - mis a jour le 20/08/2026

Sommaire

1. Objectif
2. Ou se trouvent les fichiers
3. Portee des donnees extraites
4. Lancer l'extraction (fetch_nba_data.py)
5. Mettre a jour la base (load_to_sqlite.py)
6. Si 'python' n'est pas reconnu
7. Comprendre la base de donnees
8. Construire les features et les cibles
9. Entraîner les modeles
10. Comprendre les modeles sauvegardes
11. Suivre l'etat du projet
12. Prochaines etapes

1. Objectif

Recuperer les feuilles de match NBA (box scores + play-by-play) des deux dernieres saisons, les structurer dans une base de donnees, puis entrainer des modeles pour estimer des probabilites d'evenements (points d'un joueur, double-double, pourcentage de tir...). Le detail du cadrage, des decisions et des resultats est dans projet-data-nba.md, dans ce meme dossier -- notamment le bandeau REPRISE en tete de ce fichier, toujours a jour en fin de session.

2. Ou se trouvent les fichiers

```
Cadrage/Stats/
  projet-data-nba.md      <- cadrage d'origine + journal des decisions
  README.pdf             <- ce document
  scripts/
    fetch_nba_data.py     <- extraction (nba_api -> CSV)
    load_to_sqlite.py     <- chargement (CSV -> base SQLite)
    build_features.py     <- variables predictives (features_*)
    build_targets.py      <- tables cibles (labels_*, entraînement_*)
    train_points_model.py <- modele des points (reference)
    train_doubledouble_model.py <- double-double / triple-double
    train_stat_model.py   <- reb/ast/fg3m/stl/blk/minutes (6 modeles)
    train_pct_model.py    <- FT%/FG%/3P% (3 modeles, approche taux)
    demo_pari_reel.py     <- demo sur un cas connu (Jokic)
    generate_readme.py    <- regenere ce document
    requirements.txt
  data/                  <- NON versionne dans git (trop volumineux)
```

```
raw/2024-25/, raw/2025-26/  <- CSV bruts (games_index, boxscores...)
logs/                        <- journaux d'execution de l'extraction
nba.db                       <- la base de donnees SQLite
models/                      <- NON versionne dans git
*.joblib                     <- modeles entraines et sauvegardes (S9)
```

data/ et models/ sont exclus de git (.gitignore) : volumineux et entierement regenerables en relancant les scripts (S4-S9).

3. Portee des donnees extraites

Parametre	Valeur retenue
Saisons	2024-25 et 2025-26
Types de match	Saison reguliere, Playoffs, Play-In Tournament
Box score	Traditionnel (pts/reb/ast...) + Avance (ratings, usage%, pace...)
Play-by-play	Detail de chaque action du match (tirs, fautes, remplacements...)
Volume	2 641 matchs (extraction terminee, 0 echec), ~3 fichiers par match

4. Lancer l'extraction (fetch_nba_data.py)

Ce script va chercher les données sur l'API NBA (nba_api) et les enregistrer en CSV dans data/raw/. Il est resumable : s'il est interrompu (fenêtre fermée, PC éteint, coupure réseau), il suffit de relancer la même commande -- il saute automatiquement les matchs déjà téléchargés.

Ouvrir un terminal PowerShell, se placer dans le dossier des scripts :

```
cd C:\dev\nba-pronos\Cadrage\Stats\scripts
```

Puis lancer :

```
python fetch_nba_data.py
```

Extraction des 2 saisons déjà TERMINÉE au 20/08/2026 (2 641/2 641 matchs, 0 échec) -- cette section reste utile pour une saison future ou un complément. Durée estimée pour un fetch complet : plusieurs heures (pause volontaire entre chaque appel à l'API pour éviter un blocage par NBA.com).

Pour que le script continue de tourner même si VS Code est fermé, le lancer dans une fenêtre détachée :

```
Start-Process python -ArgumentList "fetch_nba_data.py" \
  -WorkingDirectory "C:\dev\nba-pronos\Cadrage\Stats\scripts"
```

Une nouvelle fenêtre de console s'ouvre : ne pas la fermer, elle peut être minimisée.

5. Mettre à jour la base (load_to_sqlite.py)

Ce script relit tous les CSV présents dans data/raw/ et reconstruit entièrement la base data/nba.db. Rapide (quelques secondes à une minute), et sans risque : il peut être relancé autant de fois que nécessaire, y compris pendant que l'extraction tourne encore.

```
python load_to_sqlite.py
```

Un résumé s'affiche à la fin (nombre de lignes par table).

6. Si 'python' n'est pas reconnu

Utiliser le chemin complet à la place, pour n'importe quel script de ce dossier :

```
C:\Users\lenoi\AppData\Local\Programs\Python\Python312\python.exe fetch_nba_data.py
C:\Users\lenoi\AppData\Local\Programs\Python\Python312\python.exe load_to_sqlite.py
```

Cela arrive si le terminal a été ouvert avant l'installation de Python sur la machine. Fermer et rouvrir VS Code règle généralement le problème.

7. Comprendre la base de donnees

La base contient les memes informations que les CSV, mais organisees en tableaux relies entre eux plutot qu'eparpillees dans des milliers de fichiers -- ce qui permet de poser des questions directement (ex : "tous les matchs de Jokic cette saison").

Tableau	Contenu
equipes	Les 30 equipes NBA (une ligne chacune)
joueurs	Tous les joueurs apparus dans les matchs extraits
matchs	Un match par ligne : date, saison, equipes, score final
box_scores	Stats d'un joueur sur un match donne (pts, rebonds, passes...)
box_scores_advanced	Idem, stats avancees (rating offensif, usage%, pace...)
play_by_play	Chaque action du match dans l'ordre chronologique
features_equipe	1 ligne/(match, equipe) : variables predictives (S8)
features_joueur	1 ligne/(match, joueur) : variables predictives (S8)
labels_joueur	1 ligne/(match, joueur) : resultats reels a predire (S8)
entrainement_matches	1 ligne/match : features + resultat (victoire/ecart) (S8)

Les 6 premiers tableaux sont relies par des identifiants communs (game_id, player_id, team_id). Les 4 derniers (prefixe features_/labels_/entrainement_) sont DERIVES des precedents par build_features.py/build_targets.py (S8) -- reconstruits entierement a chaque execution, jamais edites a la main.

8. Construire les features et les cibles

Avant d'entrainer un modele, 2 scripts transforment les tableaux bruts (box_scores, matchs...) en variables predictives et en resultats reels a predire :

- build_features.py -- calcule les moyennes glissantes (5/10 derniers matchs), le repos, le contexte...
Regle anti-fuite stricte : chaque variable pour un match donne n'utilise QUE l'historique strictement anterieur (jamais le resultat du match qu'elle sert a predire).
- build_targets.py -- construit les tables labels_joueur
(pts/reb/ast/3-points/interceptions/contres/minutes/lancers-francs/tirs REELS, un match a la fois) et entrainement_matches (resultat de chaque match).

A lancer, dans cet ordre, apres toute mise a jour de la base (S5) :

```
python build_features.py
python build_targets.py
```

Rapide (quelques secondes), reconstruit entierement les tables a chaque execution -- sans risque, a relancer aussi souvent que necessaire.

9. Entraîner les modeles

Une fois les features et les cibles construites (S8), 4 scripts entraînent et sauvegardent les modeles (dans models/*.joblib) :

Script	Modele(s) produit(s)
train_points_model.py	points.joblib
train_doubledouble_model.py	doubledouble.joblib, tripleddouble.joblib
train_stat_model.py	reb, ast, fg3m, stl, blk, min .joblib (6 modeles)
train_pct_model.py	ft_pct, fg_pct, fg3_pct .joblib (3 modeles)

Se placer dans le dossier des scripts, puis lancer chaque script un a un :

```
cd C:\dev\nba-pronos\Cadrage\Stats\scripts
python train_points_model.py
python train_doubledouble_model.py
python train_stat_model.py
python train_pct_model.py
```

Duree : quelques secondes a 1-2 minutes par script (RandomForest sur ~56 000 lignes) -- rien a voir avec les plusieurs heures de l'extraction (S4). Chaque script ECRASE le(s) fichier(s) .joblib existant(s) : relancer un script apres une mise a jour de la base (S5-S8) reentraîne sur les donnees les plus recentes, aucune etape manuelle supplementaire.

Chaque script affiche a l'ecran ses propres metriques de calibration (MAE, R², un tableau "seuil | proba moyenne predite | taux reel | ecart") -- c'est la meme lecture a chaque fois : plus l'ecart est proche de 0%, mieux le modele est calibre. Rien a enregistrer a la main, la sortie console suffit pour verifier qu'un changement n'a pas degrade un modele.

10. Comprendre les modeles sauvegardes

Chaque fichier models/*.joblib contient un dictionnaire Python (charge avec joblib.load(...)) : le modele scikit-learn entraine, la liste des colonnes de features qu'il attend, et les parametres necessaires pour calculer une probabilite SANS reentraîner (ecart-type pour une distribution normale, ou parametres de la loi Binomiale pour un pourcentage de tir -- voir projet-data-nba.md S18).

Exemple de chargement, dans un script ou une console Python :

```
import joblib
m = joblib.load('../models/points.joblib')
print(m['target'], m['distribution'])
```

demo_pari_reel.py montre un exemple complet, sur un cas reel connu (Nikola Jokic), de chargement d'un modele + calcul d'une probabilite pour un seuil donne.

11. Suivre l'etat du projet

- Demander directement dans la conversation : je verifie et je reponds.
- Lire le bandeau REPRISE en tete de projet-data-nba.md -- toujours a jour en fin de session, resume l'etat et la prochaine etape a trancher.

- Journal complet des sessions et decisions : Cadrage/Proto/JOURNAL_SESSIONS.md (section "Projet Data NBA").

12. Prochaines etapes

- Extraction TERMINEE (2 641/2 641 matchs, 0 echec) -- plus a relancer sauf pour une saison future.
- Phase 1 (couverture des categories de paris du classeur) TERMINEE : 12 modeles construits -- points, double-double, triple-double, rebonds, passes, 3-points, interceptions, contres, minutes, FT%, FG%, 3P%.
- Decision en attente (voir bandeau REPRISE de projet-data-nba.md) : Phase 3 (affiner la calibration residuelle), Phase 4 (pont contexte en direct) ou Phase 5 (integration dans l'appli, SPEC_TECHNIQUE_PROBA_PARIS_PERSOS_V0_1.md).